



Efficient Detection of Test Interference in C Projects

Florian Eder

florian@eder.fyi

LMU Munich

Germany

Stefan Winter

sw@stefan-winter.net

Ulm University and LMU Munich

Germany

ABSTRACT

During test execution, automated software tests can interfere, i.e., their results can deviate depending on their (possibly interleaved) execution order. Such interference imposes severe restrictions on regression testing, when execution order is not or cannot be controlled for, as they can lead to non-deterministic deviations of test results giving false indication of regressions in the code base. While the phenomenon has been extensively studied for Java and Python projects, it remains unclear if or how the obtained results apply for other languages with different testing practices. Our study contributes to filling that gap by reporting results from a large-scale study on test interference in 134 C projects.

To cope with the combinatorial explosion of test execution counts when testing with all possible test orders, we propose and evaluate four novel dynamic reduction strategies for test permutations, which yield massive reductions in the number of test sequences to execute. As these strategies are specific to the resources that tests interfere on, rather than the language in which the code is written, we expect them to be useful for the study of test interference in other languages beyond C.

Based on the results obtained with these reductions, our results indicate that test order dependencies are far less common in C projects, compared to Java or Python, and that other aspects (concurrency, CPU time) more frequently threaten test result stability.

KEYWORDS

Test interference, flaky tests, C

ACM Reference Format:

Florian Eder and Stefan Winter. 2024. Efficient Detection of Test Interference in C Projects. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*, October 27–November 1, 2024, Sacramento, CA, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3691620.3694995>

1 INTRODUCTION

Automated software tests are a corner-stone of software quality assurance, as they form the basis for efficient regression testing, safe refactoring, and test-driven development. However, by automating software tests, they have entered the realm of software themselves and have become prone to the same problems that the code under

test (CUT) commonly suffers from. Intuitively, a larger test code base entails longer execution times and is more prone to defects.

While a number of practical strategies exist for limiting the execution time of automated developer tests (e.g., regression test selection or deferring expensive tests to periodic “nightly” runs), the problem of erroneous test code has been repeatedly reported as a significant problem over the past decade, in particular in the form of *flaky tests*. Flaky tests can non-deterministically pass or fail when repeatedly executed without any changes to the code and have been reported as a major impediment to regression testing and continuous integration efficacy (see [29] for an overview), because they can falsely indicate errors during development. When they do, developers may waste time hunting down bugs in their code that do not exist and are only believed to exist because of flaky tests’ spurious indication. Flaky tests are also problematic for debugging, because of their non-deterministic behavior. If a test fails consistently due to an error in the test code, the problem can be reproduced and the problematic test repaired. If, however, a test fails non-deterministically, repair success and time are affected by its (spurious) failure rate.

To this end, previous research has proposed approaches for identifying and controlling factors that trigger flaky test failures (e.g., [20, 31, 40]) and reported differing impact of these factors for different programming languages [6, 15, 17, 27]. Our study contributes to this effort by investigating the prevalence of test interference in C projects, in which test automation practices tend to differ significantly from the previously investigated languages in several aspects:

- There is no single commonly adopted test framework (cf. [30]) like JUnit for Java. As for Python, a number of different frameworks is being used, but the variety is larger [30].
- Unit tests are rare in C projects [19]. Most tests operate on the fully integrated software.
- Test executions are usually strongly isolated as individual processes¹, which restricts resource sharing across tests to shared system resources, such as files and sockets.
- From our observations, tests are mostly added to safeguard against previously encountered defects and regressions, rather than to optimize for structural coverage or mutation adequacy².

It is reasonable to assume that such differences have an effect on the prevalence of test interferences. For instance, tests that execute in isolated processes do not share heap memory by default, on

¹This is probably due to the little overhead that process isolation entails compared to multi-threading [30].

²An illustrative example from our dataset: Test files like `tests/test-rules.c` in commit 65feab41 of project VirusTotal/yara references numerous issues in the comments. Line coverage for this project has been measured as 0.68 with gcov, which is well above the median across all projects in our study (0.54), but also well below the median of the statement coverage reported for Java projects (0.78) [18].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASE '24, October 27–November 1, 2024, Sacramento, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1248-7/24/10...\$15.00

<https://doi.org/10.1145/3691620.3694995>

which they could interfere. However, they do share *system state*, such as files in the shared file system, network sockets, etc.

To study the prevalence of test interference in C projects on shared system state, we answer the following research questions in a large-scale study across 134 projects.

RQ 1: To which degree does tracking accesses to shared system state reduce interference detection overhead for C projects?

Test interference detection entails significant overheads, because all possibly interfering test sequences need to be executed. As test interference cannot occur *without* accesses to shared resources, interference detection can be omitted in these cases. We propose and evaluate four dynamic detection mechanisms for shared resource accesses to limit interference detection overheads.

RQ 2: Do tests in C projects modify shared system state?

Existing work [30] has reported initial evidence that C tests may interfere on shared file system state based on static analyses. Based on this observation of potential interference *despite comparatively strong process isolation of tests*, we extend the detection to other types of possibly shared resources across processes, such as network sockets and the POSIX environment, and systematically assess system state modifications using dynamic analyses of test executions in a large set of C projects.

RQ 3: Do C projects contain tests that interfere on shared system state?

While system state modification is a *potential* source of test interference, the modification alone is not sufficient, as we detail in Section 3.1. Hence, we investigate if test interference *manifests* in deviating test results.

RQ 4: To which degree can projects without test interference benefit from parallel execution?

Test interference is not only detrimental to test result stability (*order-dependent flaky tests* [20, 44] can change their outcome if execution order is not controlled for). A related question (“How can C projects run tests in parallel without interference?”) has been addressed before [30]. Therefore, we investigate how the published results transport to the significantly larger set of subjects in our study.

From our results, we conclude that (a) order-dependent flaky tests are less an issue for C projects than for Java or Python (only 2/13 detected interferences), (b) concurrency-dependent flaky tests account for a significant fraction of detected interferences (11/13) in our study, (c) test result deviations in C projects often result from the use of default timeout values (13/26 cases), (d) test interferences in C projects occur despite relatively low test coverage (54.5% line coverage in the median), and, maybe unsurprisingly, (e) test interference prevents test execution speedups from parallel test executions.

Most importantly, we find our proposed reduction techniques for interference detection to offer significant savings, often down to individual test executions in isolation and 0 permutations to execute. The overheads that our analyses entail are amortized by the savings with up to 30.8 h (median 97.3 s) net savings for 112

projects opposing up to 5 min (median 16.4 s) net overhead for the remaining 22 projects.

2 BACKGROUND AND RELATED WORK

In the existing literature, test interference on shared state is mostly discussed as a root cause of flaky tests. Parry et al. provide a literature survey on the topic [29], which we highly recommend for an overview. While not all flaky tests are caused by interference on shared state (e.g., deterministic implementations of non-deterministic specifications [31]), the majority are according to empirical results [15, 27]. Our notion of shared system state comprises all system resources that tests share, such as files, network sockets, and the available CPU time. In the following we use the terms *shared state* and *shared resource* synonymously, as access to a shared resource implies access to shared system state and access to shared system state implies access to one or more shared resources.

2.1 Empirical Studies

In a study of flakiness-fixing commits in Apache projects, Luo et al. [27] identified 10 recurring root causes for flaky tests, half of which involve shared resource access: CONCURRENCY describes interfering access to shared resources in parallel, interleaving test executions. TEST ORDER DEPENDENCY describes interfering access to shared resources in sequential execution. ASYNC WAIT describes situations when the results of asynchronous calls is not properly waited for. The reason why tests do not *consistently* fail in these cases is that execution times for the caller and callee vary depending on waiting time for shared resource access (CPU, locks, etc.). NETWORK describes both unavailability of a remote resource and bad management of local sockets. The latter covers interference on shared sockets. IO describes possibly interfering I/O operations (e.g., file I/O). In sum, the identified root causes involving resource dependencies account for 139/201 inspected flakiness-fixing commits (i.e., 69%) in the study. While their study targeted projects with different programming languages, the majority of the projects are written in Java and the reported C/C++ portions mostly cover auxiliary code. Except for two projects that only contain C code (APR, SpamAssassin), it is unclear whether the studied flakiness-fixing commits involve test code or CUT written in C.

In a subsequent study that focused on Python projects, Gruber et al. [15] confirmed the prevalence of many root causes that were identified by Luo et al. for Java projects. However, they found their relative frequency to differ. According to their results, TEST ORDER DEPENDENCY accounts for an impressive fraction of 59% of the identified root causes (4461/7571 tests). That is, the majority of flaky tests in their study were found to involve test interference. While the authors also identified additional (non-order-dependent) tests with root causes that potentially involve resource interference (NETWORK, IO, ASYNC WAIT, CONCURRENCY), their contribution to the prevalence of interference based flaky tests is minor compared to the bulk of order-dependent tests.

2.2 Detection Strategies

The detection strategies for flaky tests followed in existing work are mostly based on repeated executions, possibly under varying execution conditions. These conditions commonly correspond to the

hypothesized root causes of flaky tests. To detect order-dependent flaky tests, the execution order of tests within a test suite is changed and a number of different strategies to obtain test suite permutations to test with has been explored in existing research [20, 25, 41, 44]. The reduction strategies we explore in this paper are orthogonal to the previously explored prioritization techniques for the detection of test order dependencies. By reducing the number of possibly interfering tests, our strategies reduce the number of relevant tests to consider in test permutations (entire test suite permutations, based on Tuscan squares or not, or pairwise permutations).

The detection of CONCURRENCY related execution interference is covered in extensive work on concurrency testing; see [1, 11] for an overview. In contrast to this existing work, our focus is on *test interference*, i.e., specific executions of a CUT that are triggered by developer tests, rather than finding all/most possible execution interferences in a project. As noise injection [26] has been reported to be effective for stimulating the generation of diverse CPU schedules and has consequently been proposed for the detection of related flaky tests [33, 39], we employ a similar technique in our work.

An additional motivation to identify test interference is execution efficiency, as non-interfering tests can be safely executed in parallel. Schwahn et al. [30] investigated the interference of tests in C projects using a static data-flow analysis. Unlike their approach, we identify test interference using dynamic analyses, which do not depend on a certain compiler tool chain that possibly deviates from what developers are using. As our approach is significantly less intrusive, we consider it more widely applicable as witnessed by the much larger number of subjects we are able to cover in our study. However, the main motivation for our work is the identification of test interference as *defects in test code* in contrast to *minimizing regression test latencies* in their work. This allows us to use dynamic analyses, as execution efficiency considerations are of a secondary concern and can be analyzed “on hindsight”, as we do for RQ4. Moreover, our study reveals cases for which a dynamic analysis is favorable over a static analysis, because it (a) does not depend on the test code being written in the same language as the code under test and (b) does not suffer from false positives when file names are subject to complex string manipulations in the code. We further discuss these points in Section 5.4.

3 METHODOLOGY

We first detail the test interference notion in our work, followed by a description of the process for selecting the subjects of our study and their descriptive statistics. After that we present the methodology for detecting test interference and answering our research questions.

3.1 Interference Notion

Our goal is to detect test interference in C projects. By “interference” we mean that the test result differs between an isolated execution of a test and its execution with other tests, where “execution with” can either mean “in sequence” or “in parallel”. This notion of test interference covers the definitions adopted in work on order-dependent flaky tests [20–22, 44], test dependencies in general [13, 16, 24], and how parallel test executions are affected by dependencies [7, 12, 30].

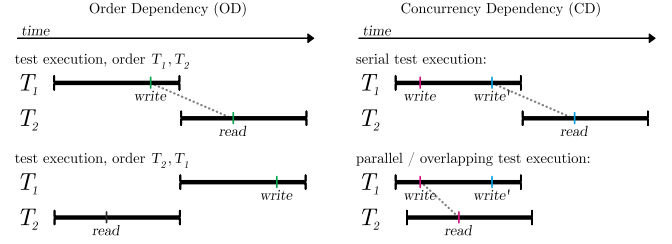


Figure 1: Illustration of the different types of interference targeted by our work.

Figure 1 illustrates the interference types covered by our notion, which we will explain below and detail in Sections 3.3 and 3.4.

Order-Dependencies (OD), left side: If tests T_1 and T_2 are executed in sequence, T_1 modifies information that the result of T_2 depends on, then a change in their execution order leads to a difference in T_2 's result. Hence, the tests interfere. Depending on which order is deemed correct, T_2 is either called a *OD-Victim* or *OD-Brittle* test in work on order-dependent flaky tests [32].

Concurrency-Dependencies (CD), right side: In the case that T_2 's result is correct when executed before T_1 and incorrect, when executed after T_1 's modification (the *OD-Victim* case), then their interference in sequential execution could be resolved if T_1 's *pollution* of the tests' shared state would be reverted by the end of T_1 's execution, as illustrated on the upper right of Figure 1. Unfortunately, this only provides a remedy for the sequential execution of the tests, as the bottom right of Figure 1 shows. If T_2 's access to the shared state occurs before T_1 resets it, the outcome of T_2 deviates from its intended outcome, whereas it does not when the access occurs after the state reset. Hence, the interference persists.

3.2 Subject Selection, Common Execution Environment, Descriptive Statistics

We base the answers to our research questions on a selection of popular C projects from GitHub, where we take GitHub “stars” as a proxy for popularity in accordance with other work in the domain (e.g., [12, 17, 20, 32, 33, 43]).

C projects employ a variety of different test tools [30] with different capabilities. To efficiently detect interference, we need to control for (a) the selection of tests to be executed, (b) the order in which tests are executed, and (c) whether they are executed in parallel or sequentially. Most test frameworks for C or C++ (e.g., GoogleTest [14] or GNU Autotest [2], as explained below) do not provide (a) or (b), two features that greatly facilitate the implementation of the detection and overhead reduction strategies proposed in this paper. We initially investigated options to influence the execution order for these testing tools, but found the required changes to be both intrusive and implementation-heavy, which would threaten the internal validity (due to intrusiveness) and the external validity (fewer subjects due to high implementation effort for supporting diverse testing tools) of our study. We consequently focus on projects using the GNU Automake test harness [38], which fulfills all of the outlined requirements. Automake [37] is part of GNU Auto-tools [36], which is a popular tool set to ensure portable builds of C projects across different platforms. To identify projects using

the Automake test harness, we search for Autotools-specific files (`Makefile.am`) in the projects' GitHub repositories. This choice also affects external validity, because our support for many subjects in our study comes at a cost of test tool diversity. We discuss the implications in Section 4.5.

As GitHub's search interface, both on the website and the API, does not accept the resulting search queries combining the search for contents in multiple files, we conduct a multi-stage search for projects that gradually filters search results by multiple criteria (programming language, stars, search hits for string patterns in multiple files). As a coarse pre-filter, we initially select all projects which

- have more than 10 stars,
- are identified by GitHub to be written in C,
- contain a file called `Makefile.am` in either the root folder or a folder called "tests" and,
- in any of these files, contain any of the strings `check_SCRIPTS`, `check_PROGRAMS`, or `check_LIBRARIES`, which indicate that test executions are indeed controlled via Automake.

Of the 566 projects matching these initial filters, we select 134 that additionally match the following characteristics:

- contain more than one test
- can build under Debian 11
- can build and execute test suites in a Podman [35] container without root permissions
- do not use *GNU Autotest* [2], as it does not respect user-specified test orders, preventing us from running test suite permutations.

We manually investigated all these projects to make sure they are not toy or course projects with short development times or few contributors. We also excluded projects that were forks of other projects with little or no development activity. The resulting list of all projects in our study is included, along with all our results and processing scripts, in our research artifact linked to in Section 7. The artifact also contains the script we used for searching subjects on GitHub.

We use containerization via *control groups* [3] and *Podman* [35] to limit possibly uncontrolled variations in the execution environments that could affect the repeated executions in our study and the reproducibility of our results. Projects for which the resulting restrictions (e.g., unmodifiable network interfaces) cause the entire test suite to fail are also excluded from the sample.

All projects in the sample are containerized in a built, but never executed state. A list of tests to execute can be passed as a parameter to the container. After the tests finish, the container returns test results and other artifacts (e.g., log files) to the host system.

Figure 2 provides an overview on descriptive statistics for the projects we selected for our study. Please note that the y-axis is on a log scale. The horizontal lines in the violins indicate the 1st, 2nd, and 3rd quartiles. The first three violins in the plot show the "stars", "forks", and "contributors". Star and fork counts were obtained via GitHub's API during our initial search for candidate projects. Contributor counts were obtained as distinct names appearing in the commit logs for the projects. The figure shows that the selected projects are popular among project users and developers with a minimum of 13 stars. In terms of forks and contributors, which

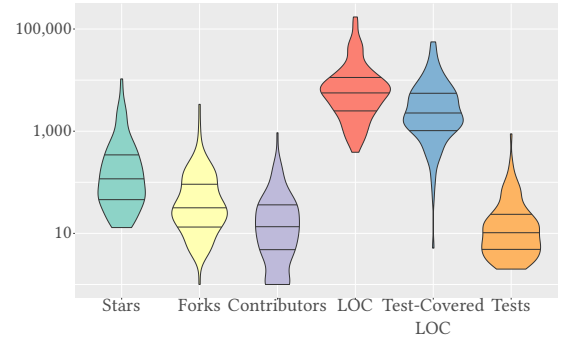


Figure 2: Distribution of popularity (stars, forks, contributors) and code measures (LOC, test-covered LOC, tests) across the subject projects in this study.

specifically indicate developers' interest in contributing to a project, the selected projects cover a large spectrum with only few contributors/forks (2/1 in the minimum for `hroptatyr/sample`) to many (591/3356 for `videolan/vlc`). The next three violins display the lines of code (LOC), the lines of code that are covered by tests, and the number of tests for the selected projects, which are discussed more in the context of RQ3 (Section 4.3).

3.3 Interference Detection: Order Dependencies

For test interference that manifests as order-dependency, we can run tests in isolation and in pairwise permutations. While some order dependencies involve more than two tests, these cases have been reported to be rare (at least for Java projects) [32, 44].

As the amount of pairwise permutations grows quadratic with the size of the test suite, we devise four different permutation reduction approaches:

- a *file descriptor* based reduction (**FD**)
- a *file descriptor* based *filtered* reduction (**FDF**)
- an *OverlayFS* based reduction (**OFS**)
- a combination of the last two (**OFSFDF**)

The FDF and OFS approaches reduce the amount of permutations compared to a naive, complete pairwise one-by-one permutation by *only permuting tests that may possibly interfere* due to observed access to shared system resources, with FDF and OFSFDF additionally removing files that are considered to be unlikely or unable to cause order-dependent flakiness (such as build artifacts). We detail the different strategies below.

3.3.1 FD Reduction. For the FD reduction approach, we execute all tests individually in isolation, logging all executed system calls using `strace` [34]. Additionally, `strace` is used to resolve the file descriptors the tests receive by the operating system as a result of any calls to `creat`, `open`, or `openat` to absolute paths on the container file system. These system calls cover *all* potential file accesses during test executions, as any file needs to be opened or created before being read or written. This is necessary to uniquely identify accessed files without knowing the current working directory, which is lost after the test exits. If a test tries to open a non-existing file, no valid file descriptor is available, causing those (non-existent) files or directories to be ignored. We also identify

Table 1: List of file types excluded by the FDF and OFSDF reduction approaches, in addition to files with identical names as a test.

Names & name patterns	Reason for exclusion
*.c, *.cpp, *.h, *.o files	build artifacts, contain source code and compiled object files
*.trs files	Automake test harness artifacts, contain test results
*.gcno, *.gda	gcov artifacts, contain coverage data
/etc, /lib, /usr, /proc, /dev, /bin, /sbin directories	system directories, no write-permission for unprivileged users
.libs/, .deps/, libtool/ directories	build artifacts, created by Automake / libtool
*.aux files, build-aux/ directories	build artifacts, contain additional Automake Makefile snippets
*.m4, *.rules, *.in, *.la files, m4/, compat/ directories	build artifacts, used by Automake
examples/ directories	test artifact, often accessed by the build system or the tests with unused write permissions

network sockets used in test executions via the `bind` system call. While we initially also tracked accesses to the POSIX environment via `getenv`, `putenv`, `setenv`, and `unsetenv`, we later decided to exclude them from our analyses, as tests execute in sibling processes and modifications to the POSIX environment are process-local, i.e., we found that tests in our study cannot interfere on their POSIX environments.

Based on the `strace`-provided information, we are able to generate a list of all files, directories, and network sockets *potentially* accessed by each test with write permissions and a second list with each file that is *potentially* accessed with read permissions. Files that are opened with both permissions are added to both lists. Only tests which potentially read file resources that are potentially modified by another test, i.e., that are opened with write permissions, are run in pairwise permutations.

It is important to note that we cannot infer from `strace`-provided information whether shared resources are *actually* accessed in test executions. We only know that the resources have been requested and with which permissions. A second limitation is that `strace` covers the entire `make check` invocation, which may also include test code compilation. Therefore, the generated access lists may be “polluted” with build artifacts, accesses to system headers, etc. For instance, in the process of compiling test code written in C, its source code file needs to be read and a resulting object file needs to be written. While these accesses appear in the `strace`-derived list of potential file accesses, they cannot indicate possible test interferences, because they are accessed during compilation rather than test execution. Hence, while the FD reduction is simple, it may indicate large numbers of *potential* interferences that actually cannot occur. We describe our solution to this issue in the following section.

3.3.2 FDF Reduction. To provide a remedy for the problem of incorrectly flagged build artifacts by the FD reduction, the FDF reduction approach reduces the permutations generated by the FD reduction by filtering out accesses to common build artifacts, such as Makefiles, compilation artifacts created by `make check` (Table 1), or any other files named identically as any test in the test suite, with the

file extension of the test or without any extension. We do this as both the build system, which is involved if tests are started via `make check`, and the test harness may access all test executables, even if only a single test is executed. If a file has a *different* extension than a test (e.g., test `example.sh` and file `example.txt`), it is not removed.

The added filter does not add significant implementation overhead and can provide significant reductions in the number of pairwise permutations to test with. On the downside, the efficacy of the reduction highly depends on the filter definition, which may be specific to the build system or even the studied projects. For example, tests for a C compiler may involve accesses to source files beyond the test builds, which could lead to missed interferences (when source files are assumed to be build artifacts and, as such, filtered out). Hence, the complexity that the filter adds lies in the required domain expertise for the projects under test and the used build system. We assume testers in practice to have the respective knowledge of the projects they are testing and the tools they are using.

3.3.3 OFS Reduction. To address the problem that resource acquisitions detected with the `strace`-based approaches (i.e., FD and FDF) do not imply persisting changes that can cause interferences, we follow an orthogonal strategy to detect *persisting changes* via OverlayFS [5]. For the OFS reduction approach, we create a list of all changed files and directories for all tests by running them individually in isolation within a Podman [35] container that uses OverlayFS.

The advantage over the FD and FDF approaches is that we can reduce the pairwise permutations to those where tests make *actually* persisting modifications to shared files, as opposed to potential modifications. However, as the container execution includes not only the test process, but the entire `make check` invocation, the OFS strategy also flags persistent modifications that are caused by test builds rather than test executions, similar to the overapproximation of relevant modification that FD provides over FDF. Moreover, while persisting *writes* can be detected reliably with OverlayFS, it cannot track which of these written files are *read* by other tests.

3.3.4 OFSDF Reduction. As the two detection strategies (potential modifications via FD/FDF and persisting modifications via OFS) are orthogonal and provide different reduction vs. detection trade-offs, we combine them in an OFSDF strategy. By filtering the results of the FDF reduction with lists of actually modified files from the OFS reduction, we are able to only permute tests which perform changes to the file system that persist after a test finishes. The obvious advantage of the strategy is an additional reduction of FDF on the basis of actually modified files. The only disadvantage is the combined overhead of collecting and traversing `strace`- and OverlayFS-based data. If tests are run in containers that utilize OverlayFS, as in our case, the additional data collection overhead over `strace` is negligible.

3.4 Interference Detection: Concurrency Dependency

Automake and `make` allow the parallel execution of the test suite by specifying the maximum number of processes and, thus, tests which can be run at the same time. We execute the test suites (1) in parallel

and (2) sequentially, both (a) with and (b) without simulated CPU noise as discussed in Section 3.4.1. To account for the inherent non-determinism of CPU schedules and any other non-deterministic effects on test results, we execute 50 repetitions for each of the four aforementioned configurations (1a, 1b, 2a, 2b).

3.4.1 Noise Injection. To provoke diverse CPU schedules, we impose CPU availability restrictions during test suite executions via BenchExec [8]. By mounting the cgroups file system of the host into the benchmark container, BenchExec’s runexec [42] utility can modify the available CPU time for the container by reducing the amount of CPU time the container’s cgroup gets (`cpu.cfs_quota_us`) to a tenth of the total CPU resource reallocation period (`cpu.cfs_period_us`). We execute the test suites 50 times in parallel with 10% of the available CPU time. We additionally run the test suites 50 times sequentially with the CPU restrictions in order to control for effects that the noise injection may impose irrespective of concurrency. We were able to run the test suites both in parallel and in sequence for 50 repetitions with and without CPU restriction for all projects, netting a total execution time of over 100 days, with the exception of *powerman* (missing all parallel runs as they trigger a blocking concurrency dependency).

3.5 Serial vs. Parallel Test Suite Run Time Benchmark

As described previously, Automake can run test suites in parallel. Using runexec [42], we record the wall-clock execution time of the test suite both in serial and parallel, for which we restrict the amount of tests that may run in parallel to 16. We chose to limit parallel executions to 16 as a typical contemporary configuration of CPU cores in desktop machines and high-end notebooks, which developers may use for running tests pre-commit. This allows an estimation of possible time savings through parallel execution. We use runexec as it can gather run time of whole process tree branches, i.e., the test harness process *and* all its descendants created by running tests, by collecting data provided by *cgroups accounting* [4], which we already use anyway. Other solutions like *GNU time* are not able to reliably collect such data from all descendants of a process.

4 RESULTS

4.1 RQ1: To which degree does tracking accesses to shared system state reduce interference detection overhead for C projects?

To answer RQ1, we execute all tests of all projects once individually to isolate changes to the test system, which we detect using *strace* and *OverlayFS*, as detailed in Section 3.3.

To ensure that the proposed reduction strategies do not miss any actual dependencies, we also executed all pairwise permutations *without any reductions*. These exhaustive permutations did not reveal any interferences missed by the reduction strategies we propose, i.e., we find them to be sound for the investigated sample.

On the basis of the identified system state modifications, we are able to reduce the number of pairwise permutations to use when probing for test interference. Using the different strategies (FD, FDF,

Table 2: Number of all pair-wise permutations (P_{max}) and after our reductions (P_{FD} , P_{FDF} , P_{OFS} , P_{OFSFDF}) for all projects with some, but not a total reduction. Bold numbers indicate the minimum number of permutations required after the reductions. Column TSP_{max} shows the number of all possible test suite permutations and TSP_{OFSFDF} the number of permutations that *would be required* with OFSFDF.

Project	P_{max}	P_{FD}	P_{FDF}	P_{OFS}	P_{OFSFDF}	TSP_{max}	TSP_{OFSFDF}
ezstream	182	182	2	182	2	8.71×10^{10}	2
flex	65280	40800	40800	25440	25440	8.13×10^{25960}	4.71×10^{284}
imagemagick	272	272	245	272	208	3.55×10^{14}	3.11×10^{10}
libbde	6	6	6	6	4	6	4
libevt	12	12	12	12	6	24	6
libevtX	20	20	20	20	8	120	8
libexe	6	6	6	6	4	6	4
libfastjson	272	272	272	272	48	3.55×10^{14}	90
libfsaps	12	12	12	12	6	24	6
libfscifs	6	6	6	6	4	6	4
libfshfs	6	6	6	6	4	6	4
libfsntfs	12	12	12	12	6	24	6
libfsrefs	6	6	6	6	4	6	4
libfvde	6	6	6	6	4	6	4
libiff	2550	2550	2057	2550	450	1.55×10^{66}	15603840
liblnk	6	6	4	6	4	6	4
liblksde	6	6	5	6	4	6	4
libmodi	6	6	6	6	4	6	4
libnsfdb	6	6	6	6	2	6	2
libolecf	12	12	12	12	6	24	6
libqcow	6	6	6	6	4	6	4
libregf	12	12	12	12	6	24	6
libscsa	6	6	6	6	4	6	4
libvhdi	6	6	6	6	4	6	4
libvmdk	6	6	6	6	4	6	4
libvshadow	2	2	2	2	1	2	1
libvslvm	6	6	6	6	4	6	4
naemon-core	1122	1122	1122	1122	2	2.95×10^{38}	2
numactl	72	72	72	72	8	362880	8
safeclib	15750	15750	15750	15750	4	2.37×10^{211}	4
xz	42	42	42	42	6	5040	6

OFS, OFSFDF), we can significantly reduce the number of relevant pairwise permutations to investigate.

FD reductions: FD removes unnecessary permutations for only one project in our study (*westes/flex*). For the remaining projects enough directories and files are opened with write permissions by *all* tests in the test suite, such that no permutations can be excluded.

FDF reductions: As we try to prevent both over-filtering by removing actual system resource dependencies between tests and creating project specific filter lists with the FDF method, we only obtain a major reduction in permutations for few projects with this method.

OFS reductions: OFS is like FD unfiltered and polluted with build artifacts, but achieves a better reduction than FD.

OFSFDF reductions: OFSFDF removes the vast majority of tests, such that the total amount of permutations required to detect OD test interference is reduced to 0 for 81 projects, i.e., *no test permutations need to be run, as tests cannot interfere in sequential executions*.

22 projects show no reduction compared to exhaustive ($n \cdot (n-1)$) pairwise permutation of all tests, i.e., the reductions have no effect. In Table 2, we list the 31 remaining projects which show some reductions, but do not drop to 0, list their respective maximum pairwise permutations without any reductions (P_{max}), and highlight the smallest number of required permutations across all reduction strategies (P_{FD} , P_{FDF} , P_{OFS} , P_{OFSFDF}) per project in bold font.

OFSFDF uniformly achieves the best reduction, except for two projects (xiph/ezstream and libyal/liblnk) for which it cannot improve over FDF and one project (westes/flex) for which it cannot improve over OFS. Adding to this observation from Table 2, the majority (60) of the 81 observed reductions to 0 permutations is only achieved with the combined OFSFDF approach and an identical reduction is only achieved by OFS alone (12 cases) or FDF (9 cases) for a total of 21 projects. Hence, we conclude that OFSFDF is the most effective reduction strategy for the projects in our study and that it offers significant savings for detecting test interferences in more than 80 % of the projects in our study compared to simpler reduction strategies or naive pairwise permutations. Due to the large number of projects, for which OFSFDF reduces the number of pairwise permutations to test with to 0, the median remaining permutations to test with is 0 and the arithmetic mean 27.5 %. Summing across all projects in our study, the reduction strategy saves the execution of 902 743 test pairs for order dependencies.

To put our results into a larger context, Table 2 also shows the reductions that the OFSFDF would achieve if the full test suite permutations were run instead of pairwise permutations. Column TSP_{max} lists the maximum number of test suite permutations, i.e., $n!$ for n tests, while the number of required test suite permutations TSP_{OFSFDF} for the reductions are computed by summing up the factorials for the number of possibly state-polluting tests that were identified for every test in the test suite. While interactions across state-polluting tests have been reported to rarely occur [32, 44], we see that our proposed reduction strategies can also offer drastic reductions in the number of full test suite permutations (often several orders of magnitude), if such extensive detection runs are desired. The results with a reduction to 0 or no reduction, which are omitted from the table, equally apply to full test suite permutations.

4.1.1 Analysis Overhead and Net Execution Time Reduction. While we observe significant reductions in pair-wise permutations with OFSFDF, we also notice from Figure 2 that the projects in our study have only 10 tests in the median. This raises the question, whether the observed reductions translate to any significant time savings, given the overheads our dynamic analyses entail. To analyze the net savings our proposed techniques offer, we record the execution time of entire test suite executions 50 times for each project and derive the *average test execution time* for each repetition. We approximate the *average test pair execution time* for each test suite execution by doubling its average test execution time. We multiply this number with (a) the number of reduced pairs resulting from our reductions and (b) the number of all test pairs as a comparison baseline. To obtain the *net savings* we sum up the analysis time for (1) creating the read and write lists for various resources and for (2) querying them to generate the reduced pairs and subtract that sum from the all-pairs execution time. For 112 out of 134 projects in our study, our reductions offer net savings between 0.56 s and 110 787.36 s (> 30 h). For the 22 projects, for which our strategies did not achieve any permutation reductions, the overhead for our analysis cannot be amortized by any savings; the *net overhead* ranges from 1.36 s to 297.96 s (< 5 min). The distribution of the net overheads and savings in seconds (log scale) are shown in Figure 3. To give a better picture how these savings relate to the test suite execution times, we provide a plot of the respective ratios in Figure 4.

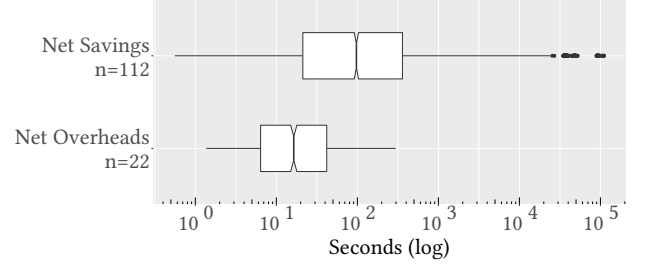


Figure 3: Net savings (for 112 projects) and overheads (for 22 projects) resulting from the proposed reduction.

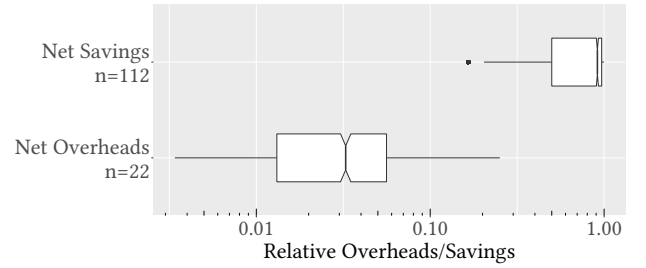


Figure 4: Relative savings (for 112 projects) and overheads (for 22 projects) resulting from the proposed reduction.

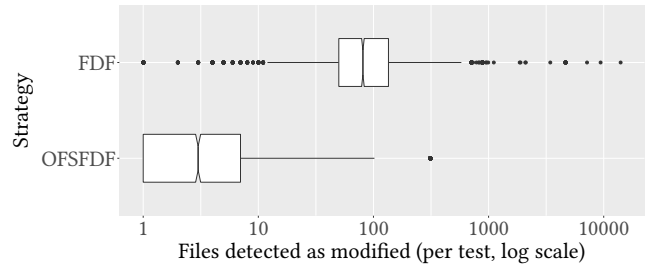


Figure 5: Distribution of the number of potential file modifications (log scale) found with the FDF and OFSFDF methods.

Findings RQ1: Tracking potential accesses via system calls (FD) does not reduce the number of permutations significantly. By removing common build artifacts (FDF) and combining it with OverlayFS-based detection (OFSFDF), we drastically reduce the number of test pairs for interference detection; to 0 in the majority of the cases and 27.5% on average across all projects in our study without missing any interferences compared to exhaustive permutations.

4.2 RQ2: Do tests in C projects modify shared system state?

Figure 5 shows the number of *possibly* (FDF) and *actually* modified files across all tests in our study. The tests that (possibly and actually) modify the system state by changing the file system span all

projects in our study. From the difference in the detected modifications, we conclude that not all file accesses with requested write permissions are acted upon, i.e., files are opened without persisting changes. Note that actual (OFSFDF) modifications only indicate *potential* test interferences, as they may pertain to system state that is not relevant to other tests. Yet, they clearly indicate that tests in C projects do persistently modify system state, which testers should be aware of to make sure these modifications do not result in interferences.

In addition to file modifications, we find 12 projects with tests which bind to TCP or UDP ports and, thereby, block other applications from using them during test execution.

Findings RQ2: Tests across all projects in our sample modify the system state. The number of files that are persistently altered beyond test execution (OFSFDF) is significantly smaller than the number of files that are transiently altered or not altered at all (FDF).

Table 3: Projects with flaky behavior due to test interference, with the category, the origin of interference, either *test code* (TC) or *code under test* (CUT), GitHub issue and its status, either *acknowledged* (ACK), *fixed* (FIX), *won't fix* (WF) or *no response* (NR).

Project	Category	Origin	Issue	Status
esnacc/esnacc-ng	OD	TC	issues/56	ACK
ptesarik/libkdumpfile	OD	TC	pull/70	FIX
strongswan/davici	CD	TC	issues/12	NR
behdad/fontconfig/	CD	CUT	(GitHub repo outdated)	n/a
farsightsec/fstrm	CD	TC	issues/69	ACK
tlwg/libdatrie	CD	TC	issues/28	NR
sustrik/libdill	CD	TC	issues/222	NR
svandenburg/libbiff	CD	TC	issues/13	NR
rsyslog/liblognorm	CD	TC	issues/371	NR
sustrik/libmill	CD	TC	(deprecated by libdill)	n/a
lwes/lwes	CD	TC	issues/8	NR
chjj/mako	CD	TC	issues/13	NR
chaos/powerman	CD	TC	issues/63	FIX
clearlinux/cve-check-tool	ETD	n/a	(user extendable timelimit)	n/a
xiph/ezstream	ETD	n/a	(user extendable timelimit)	n/a
NICMx/FORT-validator	ETD	n/a	(user extendable timelimit)	n/a
jbboehr/handlebars.c	ETD	n/a	(user extendable timelimit)	n/a
chjj/lcdb	ETD	n/a	issues/4	NR
intel/libnca	ETD	n/a	(user extendable timelimit)	n/a
alexanderchuranov/Metaresc	ETD	n/a	issues/10	WF
naemon/naemon-core	ETD	n/a	(user extendable timelimit)	n/a
cea-hpc/selfie	ETD	n/a	issues/19	NR
freewitch/sofia-sip	ETD	n/a	(user extendable timelimit)	n/a
stefanberger/swtpm	ETD	n/a	(timeout appears sensible)	n/a
troglobit/sysklogd	ETD	n/a	issues/63	ACK
videolan/vlc	ETD	n/a	(reported via mailing list)	ACK

4.3 RQ3: Do C projects contain tests that interfere on shared system state?

To answer RQ3, we rely on two different detection mechanisms as detailed in Sections 3.3 and 3.4. To lower the required number of permutations, we employ the permutation reduction approaches introduced for RQ1.

We detect a total of 13 interference-affected tests among 26 flaky tests across 26 projects. Table 3 provides an overview on the

detections. Out of 26 identified flaky tests, only two are order dependent (OD) and both are caused by interference on a shared file system dependency. In one case (ptesarik/libkdumpfile), our reduction strategies did not help and all 33 672 pairwise permutations had to be executed. In the other case (esnacc/esnacc-ng), the permutations were actually reduced to 0 executions and the order-dependency has been identified in the isolated pre-runs that are required before our reductions can be applied, as the dependency is an “OD-Brittle” case that shows when tests are run in isolation (cf. Section 3.1). The reason why it is *not* identified as a potential interference to be covered by permutations is that failed system calls for opening non-existing files do not show up in the strace log, as explained in Section 3.3.1. However, this case demonstrates that it does not affect the detection ability of our proposed reductions in any way; if a file is missing it does not show up in the logs, but if that is a problem, it will manifest in the pre-runs as an altered test verdict in isolation compared to full test suite runs.

From parallel test executions, we find 11 concurrency dependencies (CD), which are caused by either file system dependencies or ports and Unix sockets. As multiple tests try to access the same resources concurrently, race conditions and deadlocks occur.

As an interesting side-effect of the noise injection discussed in Section 3.4.1 that we utilize to stimulate diverse CPU schedules, we identify 13 *execution time dependent* (ETD) flaky tests. These tests make assumptions about execution time that are violated due to noise injection. Most ETDs manifest by exceeding explicit test timeouts, but also implicit timeouts like the expectation that recorded timestamps will remain valid for the entire duration of the test. Most explicit test timeouts are enforced and caused by the default timeout set in *libcheck*, a popular³ and commonly used unit test library. We did not notify the project maintainers about flaky tests due to those timeouts, as the timeout can be configured by the user via an environment variable (see Section 5.3 for a detailed discussion). While the flakiness can be eliminated in this way, we also note that none of the affected projects provides guidance or help in this regard; there is no specification on which hardware configuration the tests succeeded with the default timeout (or which timeout the developers specified for their runs).

Except for libcheck-related ETDs and two apparently inactive GitHub repositories (behdad/fontconfig and sustrik/libmill), we opened GitHub issues (except for farsightsec/fstrm, for which the issue has been reported before) and state the issue number and status in Table 3. We also analyzed whether the origin of the interference lies in the test code (TC) or the code under test (CUT) and find that except for one case the interference is caused by the implementation of tests. This has important implications, because the TC is not necessarily written in the same language as the CUT. For instance, the interference reported for ptesarik/libkdumpfile originates from a shell script. This highlights that language-agnostic approaches for test interference detection like ours are favorable for C projects and add value over existing approaches like the one proposed by Schwahn et al. [30], which is not able to detect the issue due to the CUT/TC language barrier.

4.3.1 Confounding Control: Test Counts. Compared to previous studies on flaky tests in Java or Python, our results reveal a relatively

³1.1k GitHub stars at the time of writing

small number of order-induced and overall test interferences. As Figure 2 shows a relatively small number of tests with a median around 10 and the test counts commonly reported in studies of Java or Python projects are considerably larger [15, 23], the number of tests may be a confounding factor affecting our findings. Fewer tests may exercise the CUT less and trigger fewer potentially interfering operations.

To assess this possible confounding, we measure the line coverage of the tests, which is also summarized in Figure 2. As previously hinted at in Section 1, the tests that ship with C projects are often system-level tests for the integrated software project rather than unit tests. This is reflected by the relative difference between LOC and test-covered LOC, which is not as dramatic as the small test counts would suggest. However, we do see varying degrees of coverage across the projects in our study, ranging from few to ten thousands of lines and relative line coverage from 1 % to 96 %, with the median line coverage being 54.5 %. Coverage could be a confounding factor if all detected interferences were in projects with high coverage: Interference-causing resource accesses in the CUT can only affect test executions if they are actually covered by the tests. However, we find only 6 of the 13 projects with interferences in Section 4.2 to have a higher-than-median test coverage. We, hence, preclude a strong effect of test coverage on our findings. Together with the observation that most interferences originate from test code, this provides initial evidence that there may be no strong link between test-covered code and proneness to test interferences in C projects. Due to the small number of interferences it is based on, this initial evidence needs to be confirmed in future studies.

As a side finding of our coverage analysis, we detected some variations in the coverage while running the test suites repeatedly for 26 projects in our study. Non-deterministic deviations in coverage have also been reported for Java projects before [18] and may indicate a potential for non-deterministic test interference. Deviating coverage implies deviating execution paths for tests, some of which may interfere while others do not. We consequently doubled the number of execution rounds for the CD detection described in Section 4.2 for the 26 projects with deviating coverage reports across repetitions, but the additional runs did not reveal additional interferences.

Findings RQ3: The number of detected test interferences is lower than in studies of Java and Python test interference. Most test interferences are concurrency-dependencies rather than order-dependencies and originate from test code (TC) rather than code under test (CUT). As a side finding, we also detect execution time dependencies, mostly caused by undocumented user settings for a timeout variable in the unit testing library *libcheck*.

4.4 RQ4: To which degree can projects without test interference benefit from parallel execution?

The result stability that interference-free tests provide is a prerequisite for safely running tests in parallel [30]. To assess the execution

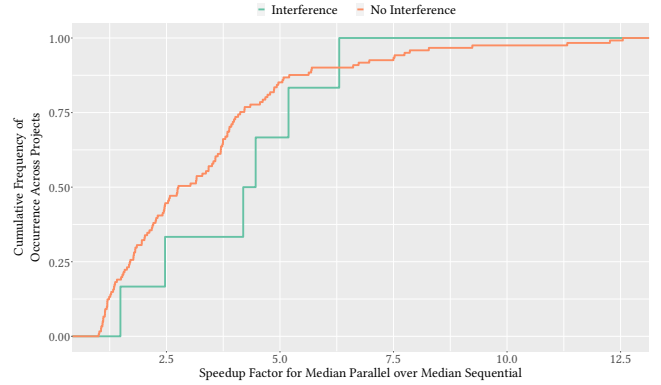


Figure 6: Distribution of the median change in execution time of the test suites while executing them in parallel in comparison to the default sequential execution.

time benefits that parallel test executions provide for the projects in our study, we measure their wall-clock time for sequential and parallel executions, except for chaos/powerman, for which the identified interference causes a deadlock that prevents parallel executions from completing. We repeat the measurements 50 times for each setting without any instrumentation for coverage or system call tracing and calculate the speedup factor from the median sequential and parallel execution times for each project. We exclude 6 projects from the analysis, for which $\geq 25\%$ of flaky failures in our measurements may significantly bias the median-based results due to premature termination because of the failures. We additionally remove between 1 and 3 flaky-failure-affected measurements for 7 projects from the analysis to avoid any such bias. As shown in Figure 6, all tests suites gain some speedup in their execution time when executed in parallel with up to 16 processes. To assess whether the *prevented* speedup from the interferences detected in our study are on the higher or lower side, i.e., what the negative impact of our detections is, we separate projects with identified interferences from those without in Figure 6. Remarkably, the cumulative distribution of the execution time improvements is *larger* for projects with interferences than for projects without, with a median speedup of 4.33 in comparison to a median speedup of 2.76. This indicates that many of the interference-affected projects in our study unfortunately miss out on above-average speedups from parallel executions.

Findings RQ4: Being able to run the test suite in parallel provides modest, but not insignificant speedups in the median. Projects with detected interferences in our study would profit more from parallel execution than projects without.

4.5 Threats to Validity

4.5.1 Construct Validity. In Section 4.3.1, we assess (limited) CUT stimulation through tests as a potential confounding factor for the few interferences we identified compared to results for projects written in other languages and we measure line coverage as a

proxy. While line coverage may be a debatable measure for CUT stimulation, we consider it to be indicative for the probability of observing test interference under the assumption that interference-inducing instructions are roughly equally distributed in the code base. We also are not able to generate coverage data for all projects, as some (jeffdaily/parasail, NICMx/FORT-validator) break in our setup when compiled with coverage generation enabled. We deem this acceptable, as it affects only two out of 134 projects.

4.5.2 Internal Validity. Due to how the `open(at)` system calls work, i.e., a valid file descriptor is only returned if the requested file exists, we are only able to detect when *existing* files are opened. If a test requires a file that does not exist and this file is the cause of interference, we theoretically miss it with our proposed strategies, but identify it during data collection because the test would fail when run in isolation with `strace`. However, if a test passes when a file does *not* exist, but fails if it exists or has certain contents and all tests that create the file execute *after* the test in the default test suite execution order, the test would have an order-dependency (it would be an *OD-Victim* [32]) that we are unable to identify.

For the FDF and OFSFDF reductions, we may risk to over-filter and, thus, be unable to detect test interferences based on filtered files. To prevent this, we chose a conservative filter as seen in Table 1, removing only file paths and files that we are quite certain to be pertaining to the build system and not the actual test executions.

As testing practices often differ for C projects compared to Java or Python, as discussed in Section 1, the observed differences may be due to these differences rather than the programming language. It is important to understand that while we denote the subjects in our study as “C projects”, we do not mean to restrict any observed differences with Java or Python projects to a direct implication of the used programming language. The observed differences may well result from differences in testing practices as we explicitly point out in our motivation for the study.

4.5.3 External Validity. As we filtered for only projects that use the Automake test harness, there may be a selection bias to C projects with a strong focus on portability. Although we applied objective criteria for subject selection, as specified in Section 3.2, 23 repositories (17.3% of our subjects) are from a single project, *libyal* [28]. Most of them are file system libraries, e.g., *libfsntfs*, a library for the NTFS filesystem. This possibly creates a bias towards file system intensive projects and coding practices of a single project/organization, although we were not able to locate any such guidelines on the GitHub-hosted wiki. As we did not identify any interference in those projects, the resulting bias for RQ2 renders our conclusion conservative. For RQ1, only a single *libyal* project’s permutations can be reduced to 0, and, thus, the exclusion of these projects would not affect our conclusion regarding the efficacy of our proposed filtering approach.

5 DISCUSSION

The results of our study quantitatively differ from those of similar studies in Java and Python [15, 20]. Of the 13 test suites with interfering tests, which is a comparatively low number, only 2 exhibit order dependencies due to manifest changes to the file system that

persist after tests finish. We provide a qualitative discussion of the identified interferences in this section.

5.1 Order Dependencies

Both identified order dependencies are file-system-based.

For `ptesarik/libkdumppfile`, a file path in `tests/xlatmap-check` assumed the directory `out` to exist, but the directory was not created as part of the test, while other tests (like `tests/addrmap-common`) create this directory in the first instruction of the test. Our pull request adds such a statement and has been accepted by the maintainer.

For `esnacc/esnacc-ng`, an ASN.1 compiler, one of the tests (`c-examples/simple/genber`) covers parts of the CUT that write data from memory into a file (`pr.ber`). Other tests (like `c-examples/simple/sbuf-ex`) then cover parts of the CUT that read the data from a file back into memory using the previously created file `pr.ber` as test data, with the test failing if the file does not exist. Our issue has been acknowledged by the maintainer, but not been addressed.

5.2 Concurrency Dependencies

The concurrency dependencies identified in our study are caused by file system dependencies, i.e., two tests modifying the same file or directory, or socket dependencies, i.e., two tests binding to or accessing the same socket. To illustrate this, we take a look at one project of each category.

`sustrik/libdill`, a library providing go-like structured concurrency in C, uses a TCP client-server architecture to communicate with concurrently running parts within tests. To do this, tests bind to port 5555 before starting the *client* co-routine with the actual test code, as shown in this example from `tests/suffix.c`:

```
54 struct ipaddr addr;
55 int rc = ipaddr_local(&addr, NULL, 5555, 0);
56 errno_assert(rc == 0);
57 int ls = tcp_listen(&addr, 10);
58 errno_assert(ls >= 0);
59 int clh = go(client());
60 int as = tcp_accept(ls, NULL, -1);
```

`ipaddr_local` populates an `ipaddr` struct with data for a local network interface, which subsequently listened on before the *client* routine is invoked in line 59. The *client* routine starts with a corresponding snippet using the hard-coded network interface:

```
31 struct ipaddr addr;
32 int cs, rc = ipaddr_remote(&addr, "127.0.0.1", 5555, 0, -1);
33 errno_assert(rc == 0);
34 int s = tcp_connect(&addr, -1);
```

The same hard-coded interface is used in several tests: `ipaddr.c`, `socks5.c`, `tcp.c`, `udp.c`, `prefix.c`. When running these concurrently, they all try to bind and connect to port 5555, as done on line 55 above, causing some of them to fail.

An example of a file system interference can be found in `rsyslog/liblognorm`, a fast log normalization library. Many tests use the same `function` to create a temporary rulebase file name:

```
78 rulebase_file_name() {
79     if [ "$1" == "x" ]; then
80         echo tmp.rulebase
81     else
82         echo $1.rulebase
83     fi
84 }
```

However, as most tests use the function without an argument, they share the default filename (`tmp.rulebase`). As tests not only

modify, but also delete their own temporary rulebase, running tests in parallel causes race conditions.

5.3 Execution-Time Dependencies

For 13 projects, we observed execution time dependencies, i.e., they assume operations to finish within a certain time frame. An example is the test `selfie_hellovar.sh` of the `cea-hpc/selfie` project, which assumes that the UNIX timestamp of the current time does not change between the start and the end of the test run:

```
34 export SELFIE_TEST="$(date +%s)"
35
36 LD_PRELOAD=${SELFIE_LIBPATH}/selfie.so ${SELFIE_TOPDIR}/test/${program} > ${SELFIE_SCRIPT}.out 2> ${SELFIE_SCRIPT}.err
37
38 grep ${SELFIE_TEST} ${SELFIE_SCRIPT}.err
```

The test assumes that, after a successful run of the tested application given in the `program` variable, it will find the timestamp previously saved to the environment variable `SELFIE_TEST` in the saved `stderr` output. If the time (in seconds) changes between the test script and the actual code under test reading the current time, the test fails.

The most common source of execution-time dependency were caused by the use of *libcheck*, a popular C testing library. By default, it enforces a 4 second timeout for each individual test. However, both the default and consciously set time limits of *libcheck* can be scaled by the user via the `CK_TIMEOUT_MULTIPLIER` environment variable.

5.4 Advantages of Dynamic OS-Level Interference Tracking

Despite the few interferences we identified in our study, we make a few observations that highlight the utility of the proposed approach.

Contrary to existing approaches, we track interference on shared resources at the operating system level. This makes our approach language- and runtime-agnostic and has proven useful in detecting interferences. The examples discussed in Section 5.1 and the file-system example in Section 5.2 are problems in shell scripts that language-specific analyses like [30] would not be able to detect.

Moreover, static analyses like the one previously proposed by Schwahn et al. [30] can cause a large number of false-positives, when matching file name strings in source code. *VLC*'s test suite, for instance, uses the string literal `/tmp/libvlc_XXXXXX` as a template for the function `vlc_mkstemp`, which mimics the standard POSIX function `mkostemp` and creates a temporary file with the last six characters replaced by random numbers and letters. As the identical template string literal is present in many tests and is fed to a function returning a file descriptor, false-positives are likely.

6 CONCLUSION AND FUTURE WORK

Observing the changes to the system state that a test performs can be a promising way to detect potential test interference (or the lack thereof) due to order dependency and concurrency dependency. By combining multiple filter approaches, it is possible to heavily reduce the number of required test runs for detection. Moreover, while we do not find noise injection to improve detection performance for concurrency-dependencies for 50 repetitions in our study, we find that running test suites with restricted computational resources is capable of identifying *execution time dependencies* affecting test

stability (at the cost of longer run times). In principle, our methods are easily adaptable for other programming languages, as long as there is a method to control which tests are executed in which order.

We observed that order-dependencies are less of a problem for C programs and libraries than for projects written in Java or Python. The most common cause for flaky behavior are concurrency dependencies, followed by execution time dependencies. The latter are commonly the result of test timeouts, especially caused by a default timeout in the test library *libcheck*.

The coverage of the code under test is not always deterministic, but can vary between repeated executions of the test suite. These fluctuations are not necessarily caused by test interference or flaky tests, but various other reasons which do not always change the externally observable behavior of the project. Future work could investigate if or to which degree such effects can be limited by limiting performance variability, e.g., by running on platforms for reliable benchmarking [9, 10].

While running test suites in parallel generally yields execution time speedups, the amount varies heavily with some projects showing almost no savings and others showing a more than 10x speedup.

Future work may improve the way potential test interference, i.e., changes to the system state, is detected. A promising idea might be to not rely on system call traces, but to detect potential system state interference directly by instrumenting file system and network interface drivers to log write and read operations.

To assess the magnitude of reductions that the techniques presented in this paper yield for other test harnesses, also in other programming languages, we plan to run previously studied projects with flaky tests in Java and Python in our experiment setup and assess system state modifications.

7 DECLARATIONS

Data Availability Statement: We publish aggregated data from our experiments along with our scripts for executing the experiments and performing the proposed reductions along with detailed documentation how to operate them on Zenodo:

<https://doi.org/10.5281/zenodo.13767954>

The container images used in our study are provided in a separate Zenodo record:

<https://doi.org/10.5281/zenodo.7935821>

Funding Statement: This work was funded in part by Deutsche Forschungsgemeinschaft (DFG) 496588242 (IdeFix) and the LMU PostDoc Support Funds.

Acknowledgments: We appreciate the extensive feedback we received from Thomas Lemberger, Marian Lingsch, Martin Spiessl, and the anonymous reviewers. We are grateful for technical support from Philipp Wendler for running our analyses on a large number of projects.

REFERENCES

- [1] Vinay Arora, Rajesh Bhatia, and Maninder Singh. 2016. A systematic review of approaches for testing concurrent programs. *Concurrency and Computation: Practice and Experience* 28, 5 (2016), 1572–1611. <https://doi.org/10.1002/cpe.3711> arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.3711>
- [2] GNU Autoconf Authors. 2023. Generating Test Suites with Autotest. https://www.gnu.org/software/autoconf/manual/autoconf-2.67/html_node/Using-Autotest.html. Accessed 2023-06-07.

- [3] Linux Kernel Authors. 2023. cgroups Documentation. <https://docs.kernel.org/admin-guide/cgroup-v1/cgroups.html>. Accessed 2024-06-07.
- [4] Linux Kernel Authors. 2023. CPU Accounting Controller. <https://docs.kernel.org/admin-guide/cgroup-v1/cpuacct.html>. Accessed 2024-06-07.
- [5] Linux Kernel Authors. 2023. OverlayFS Documentation. <https://docs.kernel.org/filesystems/overlayfs.html>. Accessed 2024-06-07.
- [6] Keila Barbosa, Ronivaldo Ferreira, Gustavo Pinto, Marcelo d'Amorim, and Breno Miranda. 2023. Test Flakiness Across Programming Languages. *IEEE Transactions on Software Engineering* 49, 4 (2023), 2039–2052. <https://doi.org/10.1109/TSE.2022.3208864>
- [7] Jonathan Bell, Gail Kaiser, Eric Melski, and Mohan Dattatreya. 2015. Efficient Dependency Detection for Safe Java Test Acceleration. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (Bergamo, Italy) (ESEC/FSE 2015)*. Association for Computing Machinery, New York, NY, USA, 770–781. <https://doi.org/10.1145/2786805.2786823>
- [8] Dirk Beyer. 2016. Reliable and Reproducible Competition Results with BenchExec and Witnesses Report on SV-COMP 2016. In *Proceedings of the 22nd International Conference on Tools and Algorithms for the Construction and Analysis of Systems - Volume 9636*. Springer-Verlag, Berlin, Heidelberg, 887–904. https://doi.org/10.1007/978-3-662-49674-9_55
- [9] D. Beyer, P.-C Chien, and M. Jankola. 2024. BENCHCLOUD: A Platform for Scalable Performance Benchmarking. In *Proc. ASE*. ACM.
- [10] Dirk Beyer, Stefan Löwe, and Philipp Wendler. 2019. Reliable Benchmarking: Requirements and Solutions. *International Journal on Software Tools for Technology Transfer* 21 (2019), 1–29.
- [11] Francesco Adalberto Bianchi, Alessandro Margara, and Mauro Pezzè. 2018. A Survey of Recent Trends in Testing Concurrent Software Systems. *IEEE Transactions on Software Engineering* 44, 8 (2018), 747–783. <https://doi.org/10.1109/TSE.2017.2707089>
- [12] Jeanderson Candido, Luis Melo, and Marcelo d'Amorim. 2017. Test Suite Parallelization in Open-Source Projects: A Study on Its Usage and Impact. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (Urbana-Champaign, IL, USA) (ASE '17)*. IEEE Press, 838–848.
- [13] Alessio Gambi, Jonathan Bell, and Andreas Zeller. 2018. Practical Test Dependency Detection. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, 1–11. <https://doi.org/10.1109/ICST.2018.00011>
- [14] Google. 2023. GoogleTest User's Guide. <http://google.github.io/googletest/>. Accessed 2023-06-07.
- [15] Martin Gruber, Stephan Lukaszczuk, Florian Kroiß, and Gordon Fraser. 2021. An Empirical Study of Flaky Tests in Python. In *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*. 148–158. <https://doi.org/10.1109/ICST49551.2021.00026>
- [16] Alex Gyor, August Shi, Farah Hariri, and Darko Marinov. 2015. Reliable Testing: Detecting State-Polluting Tests to Prevent Test Dependency. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis (Baltimore, MD, USA) (ISSTA 2015)*. Association for Computing Machinery, New York, NY, USA, 223–233. <https://doi.org/10.1145/2771783.2771793>
- [17] Negar Hashemi, Amjed Tahir, and Shawn Rasheed. 2022. An Empirical Study of Flaky Tests in JavaScript. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 24–34. <https://doi.org/10.1109/ICSME55016.2022.00011>
- [18] Michael Hilton, Jonathan Bell, and Darko Marinov. 2018. A large-scale study of test coverage evolution. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (Montpellier, France) (ASE '18)*. Association for Computing Machinery, New York, NY, USA, 53–63. <https://doi.org/10.1145/3238147.3238183>
- [19] Dmitry Ivanov, Alexey Babushkin, Saveliy Grigoryev, Pavel Iatchenii, Vladislav Kalugin, Egor Kichin, Egor Kulikov, Aleksandr Misonizhnik, Dmitry Mordvinov, Sergey Morozov, Olga Naumenko, Alexey Pleshakov, Pavel Ponomarev, Svetlana Shmidt, Alexey Utkin, Vadim Volodin, and Arseniy Volynets. 2023. UnitTestBot: Automated Unit Test Generation for C Code in Integrated Development Environments. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 380–384. <https://doi.org/10.1109/ICSE-Companion58688.2023.00107>
- [20] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakes: A Framework for Detecting and Partially Classifying Flaky Tests. In *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*. 312–322. <https://doi.org/10.1109/ICST.2019.00038>
- [21] Wing Lam, August Shi, Reed Oei, Sai Zhang, Michael D. Ernst, and Tao Xie. 2020. Dependent-Test-Aware Regression Testing Techniques. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (Virtual Event, USA) (ISSTA 2020)*. Association for Computing Machinery, New York, NY, USA, 298–311. <https://doi.org/10.1145/3395363.3397364>
- [22] Wing Lam, Stefan Winter, Angello Astorga, Victoria Stodden, and Darko Marinov. 2020. Understanding Reproducibility and Characteristics of Flaky Tests Through Test Reruns in Java Projects. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. 403–413. <https://doi.org/10.1109/ISSRE5003.2020.00045>
- [23] Wing Lam, Stefan Winter, Anjiang Wei, Tao Xie, Darko Marinov, and Jonathan Bell. 2020. A large-scale longitudinal study of flaky tests. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–29. <https://doi.org/10.1145/3428270>
- [24] Johannes Lampel, Sascha Just, Sven Apel, and Andreas Zeller. 2021. When Life Gives You Oranges: Detecting and Diagnosing Intermittent Job Failures at Mozilla. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Athens, Greece) (ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 1381–1392. <https://doi.org/10.1145/3468264.3473931>
- [25] Chengpeng Li, M. Mahdi Khosravi, Wing Lam, and August Shi. 2023. Systematically Producing Test Orders to Detect Order-Dependent Flaky Tests. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (<conf-loc>, <city>Seattle</city>, <state>WA</state>, <country>USA</country>, </conf-loc>) (ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 627–638. <https://doi.org/10.1145/3597926.3598083>
- [26] João M. Lourenço, Jan Fiedor, Bohuslav Křena, and Tomáš Vojnar. 2018. *Discovering Concurrency Errors*. Springer International Publishing, Cham, 34–60. https://doi.org/10.1007/978-3-319-75632-5_2
- [27] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (Hong Kong, China) (FSE 2014)*. Association for Computing Machinery, New York, NY, USA, 643–653. <https://doi.org/10.1145/2635868.2635920>
- [28] Joachim et al. Metz. 2023. libyal Wiki. <https://github.com/libyal/libyal/wiki/Overview>. Accessed 2024-06-07.
- [29] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2021. A Survey of Flaky Tests. *ACM Trans. Softw. Eng. Methodol.* 31, 1, Article 17 (oct 2021), 74 pages. <https://doi.org/10.1145/3476105>
- [30] Oliver Schwahn, Nicolas Coppik, Stefan Winter, and Neeraj Suri. 2019. Assessing the State and Improving the Art of Parallel Testing for C. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (Beijing, China) (ISSTA 2019)*. Association for Computing Machinery, New York, NY, USA, 123–133. <https://doi.org/10.1145/3293882.3330573>
- [31] August Shi, Alex Gyor, Owolabi Legunsen, and Darko Marinov. 2016. Detecting Assumptions on Deterministic Implementations of Non-deterministic Specifications. In *2016 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 80–90. <https://doi.org/10.1109/ICST.2016.40>
- [32] August Shi, Wing Lam, Reed Oei, Tao Xie, and Darko Marinov. 2019. iFixFlakes: A Framework for Automatically Fixing Order-Dependent Flaky Tests. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Tallinn, Estonia) (ESEC/FSE 2019)*. Association for Computing Machinery, New York, NY, USA, 545–555. <https://doi.org/10.1145/3338906.3338925>
- [33] Denini Silva, Leopoldo Teixeira, and Marcelo d'Amorim. 2020. Shake It! Detecting Flaky Tests Caused by Concurrency with Shaker. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 301–311. <https://doi.org/10.1109/ICSME46990.2020.00037>
- [34] strace Authors. 2023. strace - linux syscall tracer. <https://strace.io/>. Accessed 2023-06-07.
- [35] Containers Team. 2023. Podman Documentation. <https://podman.io/>. Accessed 2024-06-07.
- [36] GNU Autotools Team. 2023. An Introduction to the Autotools. https://www.gnu.org/software/autotools/manual/html_node/Autotools-Introduction.html. Accessed 2023-06-07.
- [37] GNU Autotools Team. 2023. Autotools Documentation. <https://www.gnu.org/software/autotools/>. Accessed 2024-06-07.
- [38] GNU Autotools Team. 2023. Autotools Test Suite Documentation. https://www.gnu.org/software/autotools/manual/html_node/Simple-Tests.html. Accessed 2024-06-07.
- [39] Valerio Terragni, Pasquale Salza, and Filomena Ferrucci. 2020. A Container-Based Infrastructure for Fuzzy-Driven Root Causing of Flaky Tests. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results (Seoul, South Korea) (ICSE-NIER '20)*. Association for Computing Machinery, New York, NY, USA, 69–72. <https://doi.org/10.1145/3377816.3381742>
- [40] Anjiang Wei, Pu Yi, Zhengxi Li, Tao Xie, Darko Marinov, and Wing Lam. 2022. Preempting Flaky Tests via Non-Idempotent-Outcome Tests. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1730–1742. <https://doi.org/10.1145/3510003.3510170>
- [41] Anjiang Wei, Pu Yi, Tao Xie, Darko Marinov, and Wing Lam. 2021. Probabilistic and Systematic Coverage of Consecutive Test-Matrix Pairs for Detecting Order-Dependent Flaky Tests. In *Tools and Algorithms for the Construction and Analysis of Systems, Jan Friso Groote and Kim Guldstrand Larsen (Eds.)*. Springer International Publishing, Cham, 270–287.
- [42] Philipp Wendler. 2023. Runexec Documentation. <https://github.com/sosy-lab/benchexec/blob/main/doc/runexec.md>. Accessed 2023-06-07.

- [43] Peilun Zhang, Yanjie Jiang, Anjiang Wei, Victoria Stodden, Darko Marinov, and August Shi. 2021. Domain-Specific Fixes for Flaky Tests with Wrong Assumptions on Underdetermined Specifications. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 50–61. <https://doi.org/10.1109/ICSE43902.2021.00018>
- [44] Sai Zhang, Darioush Jalali, Jochen Wuttke, Kıvanç Muşlu, Wing Lam, Michael D. Ernst, and David Notkin. 2014. Empirically Revisiting the Test Independence Assumption. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis* (San Jose, CA, USA) (*ISSTA 2014*). Association for Computing Machinery, New York, NY, USA, 385–396. <https://doi.org/10.1145/2610384.2610404>